VEB DASTURLAR XAVFSIZLIGI

A2. Veb ilovalarda foydalanuvchi kiritgan ma’lumotlarini xavfsiz qayta ishlash

1-VARIANT — Profil formasida kiritmalarni tekshirish, me’yorlashtirish, xavfsiz chiqarish va sanitizatsiya

Mashg‘ulot maqsadi. Talaba brauzerdagi forma tekshiruvi bilan server tekshiruvi o‘rtasidagi farqni tajribada ko‘rsatadi; foydalanuvchi kiritmalariga aniq qoidalar qo‘llaydi; oddiy matn va cheklangan HTMLni turli usullarda xavfsiz chiqaradi; xatolik haqida foydalanuvchiga ortiqcha texnik ma’lumot bermasdan javob qaytaradi.

Kutiladigan natija. Talaba noto‘g‘ri kiritmani brauzerdan tashqarida yuborib, zaif va himoyalangan ishlov berishni solishtiradi. U chegara qiymatlari, Unicode me’yorlashtirish, avtomatik HTML kodlash va ruxsat ro‘yxati asosidagi sanitizatsiyani dalil bilan tushuntiradi.

Ko‘rsatkich	Qiymat
Semestr / variant	Faqat 5-semestr / A2 / 1-variant
Tavsiya etilgan vaqt	80 daqiqa; VM, paketlar va kod oldindan tayyorlanadi
Kompyuter xonasi	30 ta Windows 11; 8 yadroli CPU, 32 GB RAM, 1 TB SSD; lokal tarmoq va internet
Virtual muhit	VMware Workstation; Ubuntu Desktop 24.04 LTS; 4 vCPU, 8 GB RAM, obrazdagi disk
Laboratoriya manzili	http://127.0.0.1:8002/ — brauzer va server Ubuntu VM ichida
Bajarish modeli	Bitta forma, weak va safe rejimlari; faylga yoki bazaga saqlash yo‘q
Baholash	10 ball; individual hisobot va qisqa himoya

0 Talaba bajaradigan dastlabki tayyorgarlik

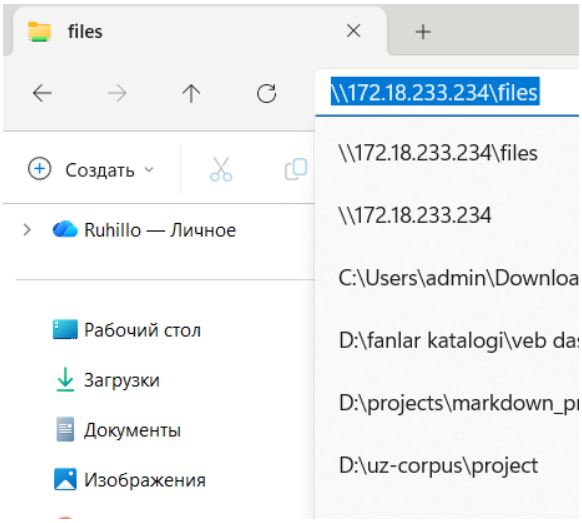
O‘qituvchi ishlaydigan tayyor VMware obrazini fayl serverga joylashtiradi. VMware o‘rnatish, obrazni nusxalash, hisoblar, proksi, paketlar, kod, sinov, snapshot va hisobotni talaba mustaqil bajaradi. Dastlabki tayyorgarlik 60–90 daqiqa yoki zarur qo‘shimcha vaqt oladi; asosiy 80 daqiqaga kirmaydi.

0.1 Fayl serverga Windows orqali kirish

Windowsda Win+R yoki File Explorer manzil satrida quyidagi SMB yo‘lni oching. Bu brauzer uchun HTTP manzil emas.

```
\\172.18.233.234\files
```

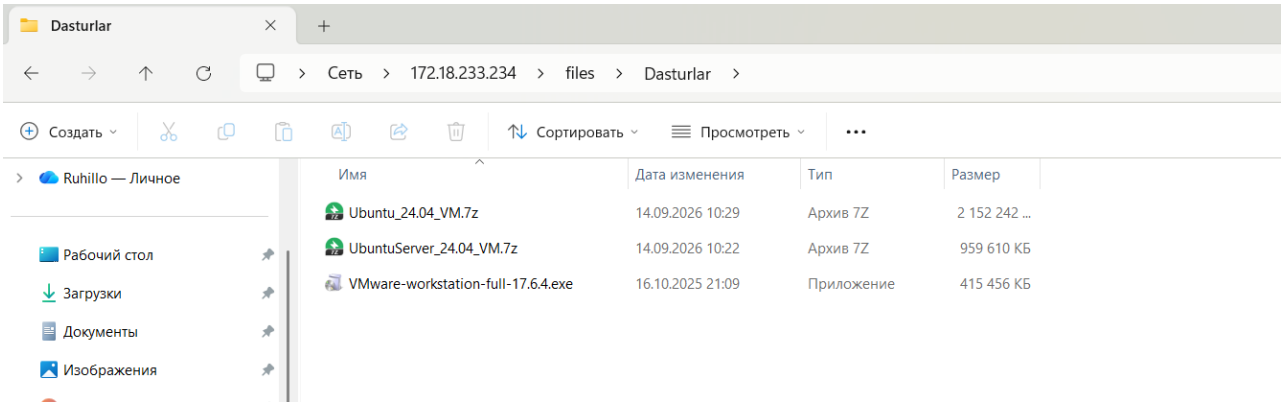
Windows Security oynasida login fileuser va parol 1234567 kiriting. Bu ma’lumotlar faqat SMB ulanishi uchun; Ubuntu yoki proksi paroli emas. Ubuntu ichida obraz bilan berilgan sudo huquqli hisobdan foydalaning.



1-rasm. Fayl serverga kirish oynasi

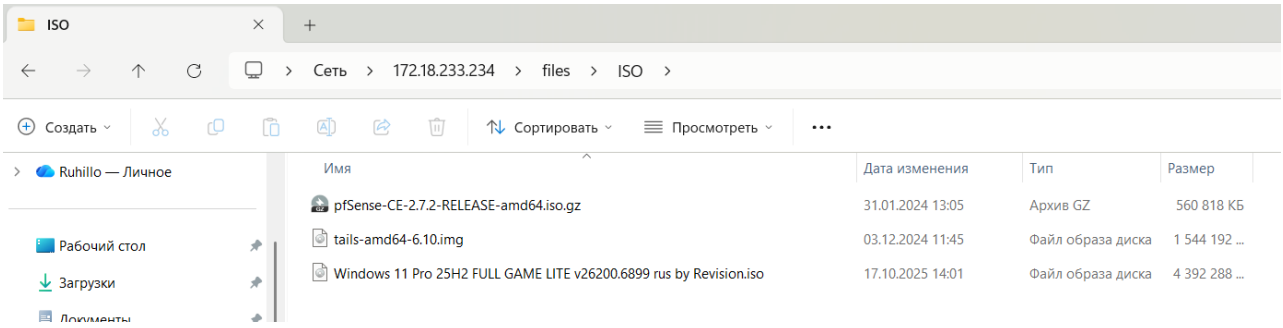
0.2 Dasturlar va tayyor obrazni tanlash

Dasturlar papkasidan VMware-workstation-full-17.6.4.exe va Ubuntu_24.04_VM.7z fayllarini Windowsning lokal diskiga nusxalang. Ubuntu_24.04_VM.7z — brauzerli Desktop; ushbu fan ishlari uchun shu obraz tanlanadi. UbuntuServer_24.04_VM.7z fayli Server muhitiga tegishli.



2-rasm. Dasturlar papkasidagi VMware va Ubuntu obrazlari

ISO papkasi serverda mavjud, lekin bu ishda Linuxni ISO orqali o‘rnatish bajarilmaydi. Boshqa operatsion tizim talab qilinsa, keyin joylashtiriladigan tegishli tayyor VMware obrazi olinadi; uning nomi oldindan taxmin qilinmaydi.



3-rasm. ISO papkasining ko‘rinishi

0.3 VMware va individual nusxani ochish

1. VMware o‘rnatgichini Windows administrator huquqi bilan ishga tushiring. O‘rnatish tugagach zarur bo‘lsa kompyuterni qayta yuklang. Obraz arxivini 7-Zip orqali D:\Veb-A2-LAB\VM papkasiga to‘liq oching; arxiv ichidan yoki tarmoq papkasidan VMni ishga tushirmang.

2. VMware → File → Open orqali ochilgan papkadagi .vmx faylini tanlang. VM nomini Veb-A2-LAB deb belgilang. Birinchi ishga tushishda “I copied it” tanlansin; har talabaning nusxasi va MAC manzili alohida bo‘ladi.

3. VM o‘chiq holatda Settings orqali 4 vCPU va 8 GB RAM belgilang. Obrazdagi virtual disk hajmini saqlang, kichraytirmang. Diskda ochilgan obraz va snapshot uchun yetarli joy qoldiring. Brauzer, curl va server bir Ubuntu VM ichida ishlaydi.

4. Ubuntu konsoliga kirib terminalni oching. Windowsdagi 127.0.0.1 VMga olib bormaydi. Quyidagi buyruqlarni Ubuntu Bashda bajaring:

```
sudo hostnamectl set-hostname veb-a2-lab
whoami
id
sudo -v
date -Is
```

0.4 Vaqtinchalik NAT va universitet proksisi

VM Settings → Network Adapter → NAT tanlang; paketlar yuklanayotganda Connected yoqilgan bo‘lsin. Bridged, port forwarding va Windows Internet Connection Sharing yoqilmaydi. 172.18.1.1:2500 — proksi manzili, VM IPsi yoki gatewayi emas.

Har yangi paket o‘rnatish terminalida quyidagi export buyruqlarini qayta bajaring. APT uchun alohida doimiy konfiguratsiya keyingi blokda berilgan.

```
export http_proxy="http://172.18.1.1:2500"
export https_proxy="http://172.18.1.1:2500"
export HTTP_PROXY="http://172.18.1.1:2500"
export HTTPS_PROXY="http://172.18.1.1:2500"
export no_proxy="localhost,127.0.0.1,::1,app.a6.test,evil.a6.test,a9.test,wrong.test"
export NO_PROXY="$no_proxy"
curl -I https://ubuntu.com
```

APT sozlamasi uchun sudo nano /etc/apt/apt.conf.d/99proxy buyrug‘ini bajaring. Mavjud fayl bo‘lsa avval mazmunini tekshiring; quyidagi uch satrni saqlang:

```
Acquire::http::Proxy  "http://172.18.1.1:2500";
Acquire::https::Proxy "http://172.18.1.1:2500";
Acquire::ftp::Proxy   "http://172.18.1.1:2500";
```

99proxy faylini saqlang va nano oynasidan chiqing. Quyidagi buyruqlar fayl mazmuni emas; ularni Ubuntu terminalida bajaring:

```
apt-config dump | grep -i proxy
sudo apt update
sudo apt install -y python3 curl nano open-vm-tools open-vm-tools-desktop
```

0.5 Firefox va mashg‘ulot paketlarini tayyorlash

Firefoxni menyudan ochib tekshiring. Mavjud bo‘lsa qayta o‘rnatmang. Firefox yo‘q bo‘lsa, APT va terminal exportlari snapd xizmatiga avtomatik o‘tmasligini hisobga olib, quyidagi xizmat konfiguratsiyasini yarating:

```
sudo apt install -y snapd
sudo mkdir -p /etc/systemd/system/snapd.service.d
sudo nano /etc/systemd/system/snapd.service.d/veb-proxy.conf
```

veb-proxy.conf fayliga faqat quyidagi uchta Environment satri va [Service] sarlavhasini yozing:

```
[Service]
Environment="HTTP_PROXY=http://172.18.1.1:2500"
Environment="HTTPS_PROXY=http://172.18.1.1:2500"
Environment="NO_PROXY=localhost,127.0.0.1,::1"
```

Faylni saqlab, muharrirdan chiqing. Endi terminalda xizmatni yangilang va Firefoxni o‘rnating:

```
sudo systemctl daemon-reload
sudo systemctl restart snapd
sudo snap install firefox
firefox --version
```

Keyingi muhitni tayyorlash bo‘limidagi apt va pip buyruqlarini NAT hali ulangan paytda bajaring. pip ishlatilgan variantda .venvni faollashtiring; sudo pip ishlatmang. requirements-lock.txt va haqiqiy versiyalarni saqlang. Yuklash xato bersa proksi/NAT va vaqtni tekshiring; paketlar tayyor bo‘lmaguncha tajribani boshlamang.

0.6 Lokal tajribaga o‘tish va dalillarni chiqarish

Barcha paketlar va kodlar tayyorlangach VMni o‘chiring. Har bir Network Adapter uchun Connected va Connect at power on belgilarini olib tashlang. Alohida laboratoriya adapteri kerak emas: barcha xizmatlar 127.0.0.1da ishlaydi. VMni qayta boshlang; Firefox → Settings → Network Settings → No proxy tanlang. curl sinovlarida --noproxy "*" ishlatiladi. A6dagi lokal domen nomlari saqlanadi.

VM o‘chiq paytda VM → Snapshot → Take Snapshot orqali Veb-A2-base yarating. Kod va konfiguratsiyani hujjatdagi boshlang‘ich rejimga tayyorlang. Snapshotga qaytishdan oldin hisobot va dalillarni Windowsga eksport qiling. Ish oxirida tiklashni talaba o‘zi bajaradi; snapshot nusxa olish o‘rnini bosmaydi.

Windowsda D:\Veb-A2-LAB\Report papkasini yarating. VM Settings → Options → Shared Folders → Always enabled → Add orqali faqat shu papkani VebA2 nomi bilan ulang. Ubuntu ichida quyidagilarni bajaring:

```
vmware-hgfsclient
sudo mkdir -p /mnt/veba2share
sudo vmhgfs-fuse .host:/VebA2 /mnt/veba2share \
  -o allow_other,uid=$(id -u),gid=$(id -g)
touch /mnt/veba2share/export-check.txt
```

vmware-hgfsclient chiqishida ulashilgan nom, Windows papkasida export-check.txt ko‘rinsin. Mountpoint oldin ulangan bo‘lsa qayta mount qilmang. Hisobot, kerakli kod va dalillarni shu katalogga nusxalang; token, cookie va yopiq kalitlarni umumiy papkaga chiqarmang.

P1 — tayyor obraz va VMware nusxasi; P2 — proksi/APT va paket versiyalari; P3 — uzilgan adapterlar hamda lokal xizmat; P4 — snapshot va Shared Folders dalillarini hisobotga qo‘shing. Kod bloklari och kulrang, muhim manzil va sozlamalar och sariq fon bilan ajratilgan.

Asosiy amaliy mashg‘ulot

Qo‘shimcha ochiq manbali dasturlar

Vosita	Vazifasi	Holati
Python 3 + venv	Dastur va alohida paket muhiti	Majburiy
Flask 3.1.3	HTTP forma, javob va xato ishlovi	Majburiy
nh3 0.3.7	HTML sanitizatsiyasi	Majburiy
Firefox + DevTools	Forma, Network va DOM tekshiruvi	Majburiy
curl, nano	HTTP sinovlari va fayl tahriri	Majburiy

A1 bilan davomiylik. A1dagi Ubuntu VMdan foydalanish mumkin; A2 alohida ~/websec/A2 papkasida va 8002-portda ishlaydi. Har kompyuter o‘z loopback manzilidan foydalanadi, shuning uchun 30 ish joyida bir xil port ziddiyat tug‘dirmaydi.

Muhim tushunchalar va ishlov berish qoidalari

Ish ketma-ketligi: so‘rovni qabul qilish; maydonlar tarkibini tekshirish; ma’lumotni kelishilgan ko‘rinishga keltirish; format va qiymat chegaralarini tekshirish; chiqarish kontekstiga mos himoyani qo‘llash. Qabul qilingan ma’lumot avtomatik ravishda HTML sifatida xavfsiz bo‘lib qolmaydi.

Tekshirish va me’yorlashtirish

Mijoz tekshiruvi xatoni tez ko‘rsatadi, server tekshiruvi esa har bir kelgan so‘rov uchun qoidani bajaradi. Brauzerdagi required, min yoki max atributlari boshqa HTTP mijozlariga majburiy cheklov bo‘la olmaydi. Sintaktik tekshiruv formatni, semantik tekshiruv qiymatning amaliy talabga mosligini tekshiradi. [1, 2]

NFC Unicode belgilarining kanonik ko‘rinishini birxillashtiradi. Bu ishda ism chetidagi bo‘sh joylar olib tashlanadi va to‘rtta apostrof shakli ASCII apostrofga almashtiriladi. Apostroflarni almashtirish NFCning avtomatik natijasi emas, laboratoriyada belgilangan alohida qoidadir. [3]

Maydon	Serverdagi talab	Me’yorlashtirish
name — ism	2–40 belgi; harflar, birikuvchi belgilar, bo‘sh joy, apostrof va defis; kamida bitta harf	NFC; ‘ ’ ‘ ’ ni apostrofga; chet bo‘sh joylarini olib tashlash
age — yosh	1–3 ta ASCII raqam; butun son 16–80 oralig‘ida	Chet bo‘sh joylarini olib tashlash; keyin int
comment — izoh	1–200 belgi; Unicode C turkumidagi belgilar taqiqlanadi, LF va tab bundan mustasno	NFC; chet bo‘sh joylarini olib tashlash
So‘rov tuzilishi	Faqat name, age, comment; har biri aynan bir marta	Noma’lum yoki takroriy maydon rad etiladi

Uzunlik Python len() orqali me’yorlashtirilgandan keyin sanaladi; bu Unicode kod nuqtalari soni, har doim ko‘zga ko‘rinadigan belgilar soniga teng emas. Ism uchun qoida o‘quv talabidir, dunyodagi barcha haqiqiy ismlarning to‘liq modeli emas. Parolni shu usulda o‘zgartirish tavsiya qilinmaydi.

Kodlash va sanitizatsiya farqi

1. Oddiy matn: A2 foydalanuvchiga aynan matn sifatida ko‘rsatiladi. HTML javobida burchak qavslar < va > ko‘rinishida kodlanadi. Flask render_template_string() bilan Jinja avtomatik kodlashidan foydalanadi. [4]
2. Cheklangan HTML: b, em, p, br teglariga ruxsat beriladi, atributlar olib tashlanadi. nh3 HTMLni tahlil qilib tozalaydi; ruxsat etilgan format saqlanadi. [5]

Kontekst cheklovi. HTML matn konteksti uchun kodlash JavaScript, CSS, URL yoki SQL uchun universal himoya emas. Sanitizatsiya qilingan fragment bu ishda faqat HTML element mazmuniga joylashtiriladi. HTMLni regex yoki “script so‘zini o‘chirish” bilan tozalash o‘rniga mos kutubxona ishlatiladi. [6]

1. Topshiriqni bajarishga ko‘rsatma

1.1. Muhitni oldindan tayyorlash

0-bo‘limdagi tayyor Ubuntu_24.04_VM.7z obrazidan foydalaning. VMda shu hujjatdagi vCPU va RAM qiymatlarini belgilang; obrazning mavjud virtual diskini saqlang. Ubuntu ichida Ctrl+Alt+T orqali Bash terminalini oching.

Ubuntu terminalida paketlarni o‘rnating va A2 uchun virtual muhit yarating:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip curl nano
mkdir -p ~/websec/A2
cd ~/websec/A2
python3 -m venv .venv
source .venv/bin/activate
python -m pip install Flask==3.1.3 nh3==0.3.7
python -m pip freeze > requirements-lock.txt
python -m pip show Flask nh3
```

Firefoxni 0-bo‘lim bo‘yicha tekshiring. Paketlar tayyor bo‘lgach NAT uziladi; requirements-lock.txt o‘rnatilgan paket versiyalarini qayd etadi.

1.2. Kodni faylga saqlash

1-ilovaning A, B va C qismlarini tartib bilan bitta a2_app.py fayliga ko‘chiring. Sahifa sarlavhasi, izoh va kolontitullarni kodga qo‘shmang. nano orqali kiritishda chekinishlarni saqlang; Ctrl+O, Enter, Ctrl+X bilan saqlang va chiqing.

```
nano a2_app.py
python -m py_compile a2_app.py
```

Sintaksis tekshiruvida xabar chiqmasin. Talaba paket va kodlarni tayyorlagach NATni uzadi va Veb-A2-base snapshotini yaratadi. Tayyorlash asosiy 80 daqiqaga kirmaydi.

1.3. Ishni boshlash va chegaralar

```
cd ~/websec/A2
source .venv/bin/activate
python a2_app.py --mode weak
```

Firefoxni Ubuntu VM ichida ochib, http://127.0.0.1:8002/ manziliga kiring. Sahifada Rejim: weak yozuvi ko‘rinsin. Server terminalini ochiq qoldiring. Alohida terminalda ss -ltnp orqali 127.0.0.1:8002 tinglayotganini tekshiring; curl buyruqlari shu ikkinchi terminalda bajariladi.

Faqat o‘quv muhiti. weak rejimi ataylab yetarli tekshiruvsiz ishlaydi. Uni faqat o‘z VMingizda sinang; serverning 127.0.0.1 bog‘lanishini o‘zgartirmang. Dasturda haqiqiy akkaunt, sessiya, shaxsiy ma’lumot yoki bazaga yozish yo‘q. Flask ishlab chiqish serveri va ushbu demo tayyor ishlab chiqarish tizimi hisoblanmaydi.

1.4. Brauzer tekshiruvi va uning chegarasi

F12 → Network panelini oching. Ism: Ali, yosh: 20, izoh: Salom qiymatlari bilan Tekshirish tugmasini bosing. /preview uchun POST so‘rovi va 200 statusni toping. Form Data maydonlarida yuborilgan qiymatlarni kuzating. Natija sahifasidagi Formaga qaytish havolasi bilan qayting.

Yoshni 150 ga o‘zgartirib, yana yuboring. Brauzer max="80" sabab xabar ko‘rsatishi va yangi POST yubormasligi kerak. Shu holatni Network panelidagi kuzatuv bilan asoslang. Bu serverning 150 qiymatini rad etganini isbotlamaydi. [2]

1.5. Brauzerni chetlab o‘tib so‘rov yuborish

Ubuntu ichidagi ikkinchi terminalda xuddi shu qiymatni to‘g‘ridan-to‘g‘ri yuboring. curl HTML formadagi max atributini tekshirmaydi:

```
curl --noproxy "*" -i http://127.0.0.1:8002/preview \
  --data-urlencode "name=Ali" \
  --data-urlencode "age=150" \
  --data-urlencode "comment=Salom"
```

Kutiladigan natija: weak rejimida 200 va natija HTMLida Yosh: 150. Mijoz cheklovi ishlagan, serverdagi yosh diapazoni tekshiruvi esa yo‘q. Statusni va qabul qilingan qiymatni dalil sifatida saqlang.

Brauzer orqali ham tasdiqlash uchun formaga qayting. Inspector oynasida age nomli input elementini toping, faqat max="80" atributini vaqtincha olib tashlang. 150 bilan yuboring: weak rejimida natija ochiladi. Keyingi bosqichdan oldin / sahifasini yangilab, max atributini asl holatiga qaytaring.

1.6. Tekshirilmagan HTMLni chiqarish oqibati

weak rejimida yoshni 20, ismni Ali qilib, izohga quyidagini kiriting:

```
<b>A2</b>
```

Natijada “Oddiy matn sifatida” qismida teglar matn ko‘rinishida chiqish o‘rniga A2 qalin yoziladi. Inspector orqali plain elementining ichida haqiqiy b elementi paydo bo‘lganini kuzating. Bu faol hujum talab qilmasdan, foydalanuvchi kiritmasi HTML sifatida talqin qilinayotganini ko‘rsatadi.

```
plain = Markup(comment)  # weak rejimdagi ataylab xavfli satr
```

Markup bu qiymatni “allaqachon xavfsiz HTML” deb belgilaydi; u o‘zi sanitizatsiya qilmaydi. Tekshirilmagan foydalanuvchi matniga shu belgi qo‘yilishi avtomatik kodlashni bekor qiladi. Bu usul safe rejimdagi oddiy matn yo‘lida ishlatilmaydi. [4]

Dalilni to‘g‘ri talqin qiling. Qalin matn chiqishi o‘zi JavaScript bajarilganini isbotlamaydi. U matn–HTML chegarasi buzilganini ko‘rsatadi. A2ning maqsadi ushbu chegarani to‘g‘ri boshqarish; XSS turlarini keng tekshirish keyingi mavzularda bajariladi.

1.7. Server tekshiruvi mavjud rejimga o‘tish

Server terminalida Ctrl+C bosing va o‘sha virtual muhitda qayta ishga tushiring:

```
python a2_app.py --mode safe
```

Brauzerda / sahifasini yangilang; Rejim: safe yozuvi chiqsin. 1.5-banddagi age=150 so‘rovini qayta yuboring. Endi 400 status va yosh oralig‘i haqida xabar chiqishi kerak. age=20 qiymati esa 200 qaytaradi. Brauzerning cheklovi olib tashlansa ham server qoidasi amal qiladi.

1.8. Chegara qiymatlarini tekshirish

Quyidagi sikl har bir qiymat uchun faqat HTTP statusni chiqaradi. Keyin kamida bitta 400 javobini curl -i bilan ochib, xato izohini ham tekshiring:

```
for age in 15 16 80 81 abc 20.0; do
  printf "age=%s status=" "$age"
  curl --noproxy "*" -s -o /dev/null -w "%{http_code}\n" \
    http://127.0.0.1:8002/preview \
    --data-urlencode "name=Ali" \
    --data-urlencode "age=$age" \
    --data-urlencode "comment=Salom"
done
```

Kutiladigan ketma-ketlik: 400, 200, 200, 400, 400, 400. 15 va 81 — semantik chegaradan tashqari; abc va 20.0 — ushbu butun son formati talabiga mos emas. Qoidada faqat ASCII 0–9 qabul qilinadi; boshqa raqam yozuvlari avtomatik almashtirilmaydi.

1.9. Ismni me’yorlashtirish

Quyidagi so‘rovdagi name qiymati boshida va oxirida ikkita bo‘sh joy bor; ism ichidagi belgi U+02BB (‘). Buyruqni UTF-8 matn sifatida bajaring:

```
curl --noproxy "*" -i http://127.0.0.1:8002/preview \
  --data-urlencode "name=  O‘tkir  " \
  --data-urlencode "age=20" \
  --data-urlencode "comment=Salom"
```

Kutiladigan natija: 200; brauzerda ko‘rsatiladigan ism O'tkir bo‘ladi, chetdagi bo‘sh joylar olib tashlanadi. curl qaytargan HTMLda apostrof ' kabi entity bilan kodlanishi mumkin — bu qiymat buzilgani emas. Formada ham shu ismni yuborib tekshiring. Keyin Ali7 va A qiymatlarini sinang: ikkalasi 400 qaytaradi.

1.10. Takroriy maydonni rad etish

```
curl --noproxy "*" -i http://127.0.0.1:8002/preview \
  -d "name=Ali&age=20&age=150&comment=Salom"
```

Kutiladigan natija: 400 va maydonlarni bittadan yuborish talabi. safe rejimi birinchi yoki oxirgi age qiymatini yashirincha tanlamaydi. Bu qoida so‘rovning turli komponentlar tomonidan turlicha talqin qilinishini kamaytiradi.

1.11. Oddiy matnni xavfsiz chiqarish

safe rejimida ism: Ali, yosh: 20, izoh: A2 qiymatlarini formada yuboring. Natija sahifasida bir kiritmaning ikki xil chiqarilishini ko‘rasiz:

3. “Oddiy matn sifatida” qismida A2 teglar bilan birga matn bo‘lib chiqadi; Inspector orqali plain ichida b elementi yo‘qligini tekshiring.
4. “Cheklangan HTML sifatida” qismida A2 qalin chiqadi; rich ichida b elementi bor. Bu teg aynan ruxsat ro‘yxatiga kiritilgan.
5. “Sanitizatsiyadan keyingi HTML kodi” qismida tozalangan fragmentning o‘zi matn sifatida ko‘rsatiladi. Network → Response oynasidagi kod bilan solishtiring.

```
plain = comment
# RESULT ichidagi {{ plain }} avtomatik HTML kodlashdan otadi.
```

Oddiy matn uchun kiritmani oldindan qo‘lda HTML entitylarga aylantirish shart emas: chiqarish vaqtida avtomatik kodlash ishlaydi. Bir qiymatni qo‘lda kodlab, so‘ng yana autoescape orqali chiqarish ikki marta kodlanishga olib kelishi mumkin. [4, 6]

1.12. HTML sanitizatsiyasini tekshirish

safe rejimida izohga quyidagi zararsiz tekshiruv fragmentini yuboring:

```
<b onclick="void(0)">Salom</b><img src=x>
```

“Oddiy matn sifatida” maydonida barcha belgilar matn bo‘lib ko‘rinadi. “Cheklangan HTML sifatida” maydonida esa qalin Salom qoladi. “Sanitizatsiyadan keyingi HTML kodi” qismidagi kutiladigan qiymat:

```
<b>Salom</b>
```

Inspector → rich ichida onclick atributi ham, img elementi ham yo‘qligini ko‘rsating. Butun javob HTMLida onclick so‘zi qolishi mumkin, chunki oddiy matn qismida asl kiritma kodlangan holda ko‘rsatilmoqda; faqat satr qidiruvi bilan sanitizatsiyani baholamang.

```
clean = nh3.clean(comment, tags={'b', 'em', 'p', 'br'},
                  attributes={}, strip_comments=True)
```

nh3 HTML fragmentini tahlil qiladi, ruxsat etilmagan teg va atributlarni olib tashlaydi. Markup(clean) faqat shu tozalangan fragmentni rich maydonida HTML sifatida chiqarish uchun ishlatiladi. Tozalashdan keyin fragmentga tekshirilmagan HTML qo‘shilmaydi. [5]

Talaba mustaqil bajaradi

Izoh maydonida Sinov va <u>Sinov</u> ni alohida yuboring. Qaysi format saqlanganini va qaysi teg olib tashlanganini taxmin qiling, keyin DOM bilan tekshiring. Oddiy matn chiqarilishida ikkala kiritma ham literal ko‘rinishda qolishini qayd eting.

Bu sinovda CSP qo‘shilmagan. Kuzatilayotgan himoya natijasi aynan server tekshiruvi, Jinja kodlashi va nh3 sanitizatsiyasiga tegishli. Ularning mavjudligi autentifikatsiya, avtorizatsiya yoki CSRF himoyasi ham borligini anglatmaydi.

1.13. Xatoliklarni xavfsiz boshqarish

safe rejimida age=abc yoki bo‘sh comment yuboring. Kutiladigan javob — 400 va foydalanuvchi tuzatishi mumkin bo‘lgan izoh. Keyin http://127.0.0.1:8002/lab-error manzilini oching yoki quyidagi buyruqni bajaring:

```
curl --noproxy "*" -i http://127.0.0.1:8002/lab-error
```

Bu maxsus o‘quv endpointi boshqariladigan ichki xatoni yuzaga keltiradi. Javob statusi 500 bo‘ladi; sahifada faqat umumiy xabar va 12 belgili qayd raqami chiqadi. Server terminalida o‘sha raqam bilan diagnostik yozuv va traceback paydo bo‘ladi. Ikkala joydagi raqamni solishtiring. Brauzer javobida fayl yo‘li yoki traceback bo‘lmasligi kerak. [7]

Xato tafsiloti server diagnostikasiga tegishli; foydalanuvchiga esa kuzatish raqami kifoya. Ilova debug=False va use_reloader=False bilan ishlaydi. /lab-error faqat laboratoriya uchun kiritilgan va haqiqiy xizmatga ko‘chirilmaydi.

1.14. Boshqariladigan nosozlik: yosh chegarasi buzilishi

Asl fayldan nusxa oling. Serverni Ctrl+C bilan to‘xtating, a2_app.py faylining boshidagi MAX_AGE qiymatini vaqtincha 800 qilib o‘zgartiring. MIN_AGE 16 bo‘lib qoladi:

```
cp a2_app.py a2_app.original.py
nano a2_app.py
# Vaqtincha: MIN_AGE, MAX_AGE = 16, 800
python a2_app.py --mode safe
```

age=150 so‘rovini yana curl bilan yuboring. Endi 200 chiqadi, garchi brauzer formasida max="80" saqlanib qolgan bo‘lsa ham. Bu server kodidagi noto‘g‘ri qoida mijozdagi to‘g‘ri atribut bilan tuzatilmasligini ko‘rsatadi.

Serverni to‘xtating, MAX_AGE qiymatini 80 ga qaytaring va qayta ishga tushiring. age=80 → 200, age=81 → 400 va age=150 → 400 bo‘lishini tekshiring. Natija faqat qabul/rad etish bo‘yicha emas, biznes qoidasi bilan mosligi bo‘yicha baholanadi.

1.15. Hajm cheklovi — ixtiyoriy qo‘shimcha

Ilova HTTP so‘rovi tanasini 8192 bayt bilan cheklaydi. 9000 ta ASCII belgili izoh yuborilganda 413 qaytishi kerak; bu izohning 200 belgilik maydon cheklovidan alohida chegaradir. Quyidagi blokni terminalga to‘liq kiriting:

```
python3 - <<'PYCODE'
import urllib.parse, urllib.request, urllib.error
data = urllib.parse.urlencode(
    dict(name='Ali', age='20', comment='A' * 9000)).encode()
try:
    urllib.request.urlopen(
        'http://127.0.0.1:8002/preview', data=data)
except urllib.error.HTTPError as e:
    print(e.code)
PYCODE
```

Asosiy ish yakunida hisobot va kod nusxasini saqlang; serverni Ctrl+C bilan to‘xtating. Snapshotga qaytishdan oldin hisobot nusxasi VMdan tashqarida saqlanganiga ishonch hosil qiling.

2. 1-variant topshirig‘i

Vaziyat: o‘quv profil formasida ism, yosh va izoh olinadi. Brauzer tekshiruvi yetarli deb hisoblangan, izoh esa to‘g‘ridan-to‘g‘ri HTMLga chiqarilgan. Siz shu ikkita muammoni ko‘rsatishingiz va himoyalangan ishlov berishni tekshirishingiz kerak.

6. weak rejimida brauzer cheklovini va age=150 qiymati curl orqali qabul qilinishini ko‘rsating.
7. safe rejimida server tekshiruvi, me’yorlashtirish va takroriy maydon rad etilishini tekshiring.
8. Oddiy matn chiqarilishi bilan cheklangan HTML sanitizatsiyasini solishtiring; DOM dalili keltiring.
9. 400 va 500 xatolarini ajrating; 500 javobidagi qayd raqamini server jurnali bilan bog‘lang.
10. MAX_AGE nosozligini kiriting, aniqlang va tuzating; yakuniy chegaralarni qayta tekshiring.

3. To‘ldiriladigan natija jadvali

Boshqa ko‘rsatilmagan maydonlar: name=Ali, age=20, comment=Salom. “Haqiqiy natija” ustuniga o‘z status/natijangizni, dalil nomini esa hisobotga yozing.

Sinov	Rejim	Kutiladigan natija	Haqiqiy natija
Brauzer: age=150	weak	POST yuborilmaydi	_____
curl: age=150	weak	200, qiymat qabul	_____
age=15 / 16	safe	400 / 200	_____
age=80 / 81	safe	200 / 400	_____
age=abc / 20.0	safe	400 / 400	_____
name= O‘tkir	safe	200, O'tkir	_____
name=Ali7 / A	safe	400 / 400	_____
age ikki marta	safe	400	_____
comment=A2	safe	Plain: literal; rich: qalin	_____
onclick + img fragmenti	safe	Rich: faqat Salom	_____
/lab-error	safe	500, qayd raqami bor	_____
MAX_AGE=800; age=150	safe	200 — noto‘g‘ri qoida	_____
Tiklangan; age=80 / 81	safe	200 / 400	_____

Mustaqil tahlil uchun qisqa qayd

Ism va izoh maydonlarining qoidasi nega bir xil emas? _____
Kodlash va sanitizatsiya natijasining farqi: _____
Mijoz va server tekshiruvi farqi: _____

4. Topshiriladigan dalillar

11. D1 — VM va paket versiyalari; weak rejimida brauzer age=150ni to‘xtatishi hamda curl 200 olishi.
12. D2 — safe rejimdagi chegara qiymatlari; apostrof/bo‘sh joy me’yorlashtirilishi; takroriy age uchun 400.
13. D3 — A2 kiritmasining weak va safe natijasi; plain va rich elementlarining DOM farqi.
14. D4 — onclick va img olib tashlangan toza fragment; hamda <u> mustaqil sinovi.
15. D5 — 500 javobi va unga mos jurnal raqami; MAX_AGE nosozligi hamda tiklashdan keyingi statuslar.

Hisobot nomi: A2_1_Guruh_Familiya.docx. Natija jadvali, kamida D1–D5 dalillari va 5–8 gaplik xulosa kiritilsin. Har skrinshot/terminal parchasi ostida uning nimani isbotlashini 1–2 gap bilan yozing. Yakuniy a2_app.py fayli MAX_AGE=80 bilan saqlansin.

5. Tavsiya etilgan 80 daqiqalik ish taqsimoti

Bosqich	Vaqt	Natija
Muhit va boshlang‘ich so‘rov	8 daqiqa	Ishlayotgan weak rejimi
Mijoz cheklovi va zaif HTML chiqarish	12 daqiqa	D1 va dastlabki D3
Server tekshiruvi va me’yorlashtirish	18 daqiqa	Chegaralar, D2
Kodlash va sanitizatsiya	15 daqiqa	D3–D4
Xatoliklar, MAX_AGE nosozligi va tiklash	15 daqiqa	D5
Hisobot, xulosa va yakunlash	12 daqiqa	Topshirishga tayyor ish
Jami	80 daqiqa	Tayyor VM va kod bilan

6. Baholash mezoni — 10 ball

Mezon	Ball	To‘liq ball sharti
Muhitni ishga tushirish	1,0	VM, paketlar va lokal server
Mijoz/server tekshiruvi farqi	1,5	Bir xil kiritma bilan zaif va safe taqqoslashi
Qoidalar va chegara sinovlari	2,0	Yosh, ism, takroriy maydon, me’yorlashtirish
Xavfsiz chiqarish va sanitizatsiya	2,0	Plain/rich farqi, teg/atribut dalili
Xatolik va nosozlikni tuzatish	2,0	400/500, jurnal raqami, MAX_AGE tiklash
Hisobot va qisqa himoya	1,5	Jadval, izohlangan dalillar, xulosa
Jami	10,0	

Mezon natija va izoh bilan to‘liq bajarilganda to‘liq ball beriladi; qisman bajarilsa ballning yarmigacha, bajarilmasa 0 ball. Ixtiyoriy 413 sinovi bajarilmagani uchun ball kamaytirilmaydi. Paket o‘rnatish va kodni terish zarur bo‘lsa, darsga qo‘shimcha vaqt ajratiladi.

7. Ko‘p uchraydigan muammolar

Belgi	Sabab / yechim
ModuleNotFoundError	cd ~/websec/A2 va source .venv/bin/activate bajaring; python -m pip show Flask nh3 bilan shu muhitni tekshiring.
Externally managed environment	Paketni tizim Pythoniga o‘rnatmang; python3 -m venv .venv yarating va faollashtiring.
Port band yoki ulanish yo‘q	ss -ltnp bilan 8002ni tekshiring. Eski server terminalida Ctrl+C bosing. Brauzer Ubuntu VM ichida ochilgan bo‘lsin.
Kod o‘zgardi, natija o‘zgarmadi	use_reloader=False: serverni Ctrl+C bilan to‘xtatib qayta ishga tushiring; sahifani yangilang.
Brauzer 400/500 ko‘rsatmadi	Status sahifa matnida yozilmasligi mumkin. Network yoki curl -i orqali HTTP statusni ko‘ring.

Belgi	Sabab / yechim
Sanitizatsiya bo‘lsa ham onclick so‘zi bor	Butun javobni emas, rich elementini tekshiring. Asl fragment plain ichida xavfsiz matn sifatida qoladi.
Name apostrofi boshqacha chiqdi	Bu variantda to‘rtta apostrof shakli ASCIIga aylantiriladi. HTML entity ko‘rinishi bilan brauzer matnini adashtirmang.
SyntaxError / chekinish xatosi	Ilova qismlarini tartib bilan bitta faylga ko‘chiring; 4 bo‘sh joyli chekinish va oddiy qo‘shtirnoqlarni saqlang; py_compile bajaring.

8. O‘qituvchi uchun qisqa himoya savollari

16. Nega HTMLdagi max="80" server tekshiruvining o‘rnini bosmaydi?
17. age=abc va age=81 rad etilishi qaysi ikki xil tekshiruvga misol?
18. NFC bilan apostrofni almashtirish orasida qanday farq bor?
19. Nega A2 plain ichida matn, rich ichida format sifatida chiqadi?
20. Markup funksiyasi sanitizatsiya qiladimi? Uni qaysi qiymatga qo‘llash mumkin?
21. Nega 500 javobida traceback emas, qayd raqami beriladi?

9. Talaba uchun yakuniy qayd

Band	Qayd
Talaba F.I.Sh. / Guruh	_____ / _____
Sana / VM nomi	_____ / _____
Olingan ball / O‘qituvchi imzosi	_____ / 10 _____

Talabaning tayyorgarlik nazorati: weakda age=150 → 200; safe holatda 16/80 → 200 va 15/81 → 400; rich ichida onclick/img yo‘q; /lab-error → 500 va umumiy xabar. Ish davomida MAX_AGE=800 nosozligi kiritilsa, yakunda 80ga qaytarilganini tekshiring.

1-ilova. Laboratoriyaning to‘liq kodi

Fayl: a2_app.py. Quyidagi A, B va C qismlar bitta faylni tashkil qiladi. Faqat kod satrlarini ko‘chiring; sarlavhalar va manbalarni qo‘shmang. Flask hamda nh3 oldindan o‘rnatilgan bo‘lishi kerak. Koddagi weak tarmog‘i ataylab zaif taqqoslash namunasi hisoblanadi.

A qism — importlar, forma va natija shablonlari

```
import argparse
import re
import unicodedata
import uuid
import nh3
from flask import Flask, request, render_template_string
from markupsafe import Markup
from werkzeug.exceptions import HTTPException

app = Flask(__name__)
app.config['MAX_CONTENT_LENGTH'] = 8192
app.config['LAB_MODE'] = 'safe'
MIN_AGE, MAX_AGE = 16, 80
FORM = '''<!doctype html><html lang="uz"><meta charset="utf-8">
<title>A2 - Kiritmalar</title><h1>A2: profilni tekshirish</h1>
<p>Rejim: {{ mode }}</p>
<form method="post" action="/preview">
<p>Ism: <input name="name" required minlength="2"
 maxlength="40" value="Ali"></p>
```

```
<p>Yosh: <input name="age" type="number" required
 min="16" max="80" value="20"></p>
<p>Izoh: <textarea name="comment" required
 maxlength="200">Salom</textarea></p>
<button type="submit">Tekshirish</button></form></html>'''
RESULT = '''<!doctype html><html lang="uz"><meta charset="utf-8">
<title>A2 - Natija</title><h1>Tekshiruv natijasi</h1>
<p>Rejim: {{ mode }}</p><p>Ism: {{ name }}</p>
<p>Yosh: {{ age }}</p><h2>Oddiy matn sifatida</h2>
<pre id="plain">{{ plain }}</pre>
<h2>Cheklangan HTML sifatida</h2><div id="rich">{{ rich }}</div>
<h2>Sanitizatsiyadan keyingi HTML kodi</h2>
<pre id="clean-source">{{ clean_source }}</pre>
<p><a href="/">Formaga qaytish</a></p></html>'''
ERROR = '''<!doctype html><meta charset="utf-8"><title>A2 - Xato</title>
<h1>So'rov bajarilmadi</h1><p id="error">{{ message }}</p>
<p><a href="/">Formaga qaytish</a></p>'''
APOSTROPHES = str.maketrans({c: "'" for c in "’‘ʻ"})

def normal_name(value):
    return unicodedata.normalize('NFC', value).translate(
        APOSTROPHES).strip()

def fail(message, status=400):
    return render_template_string(ERROR, message=message), status
```

1-ilova davomi. B qism — server tekshiruvi

```
@app.after_request
def headers(response):
    response.headers['Cache-Control'] = 'no-store'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    return response

@app.get('/')
def index():
    return render_template_string(FORM, mode=app.config['LAB_MODE'])

@app.post('/preview')
def preview():
    form = request.form
    if app.config['LAB_MODE'] == 'weak':
        name = form.get('name', '')
        age = int(form.get('age', ''))
        comment = form.get('comment', '')
        clean = comment
        plain = Markup(comment)  # Deliberately unsafe lab mode.
    else:
        fields = {'name', 'age', 'comment'}
        if set(form) != fields or any(
                len(form.getlist(k)) != 1 for k in fields):
            return fail('Uchta maydonni aynan bir martadan yuboring.')
        name = normal_name(form['name'])
        age_text = form['age'].strip()
        comment = unicodedata.normalize('NFC', form['comment']).strip()
        if not (2 <= len(name) <= 40) or not all(
                c.isalpha() or unicodedata.category(c).startswith('M')
                or c in " '-" for c in name) or not any(
                c.isalpha() for c in name):
            return fail('Ism 2-40 belgi: harf, boshliq, apostrof, defis.')
        if not re.fullmatch(r'[0-9]{1,3}', age_text):
            return fail('Yosh ASCII raqamlari bilan yozilishi kerak.')
        age = int(age_text)
        if not MIN_AGE <= age <= MAX_AGE:
            return fail('Yosh 16-80 oraligida bolishi kerak.')
        if not 1 <= len(comment) <= 200 or any(
```

```
                unicodedata.category(c).startswith('C')
                and c not in '\n\t' for c in comment):
            return fail('Izoh 1-200 belgi; boshqaruv belgilarisiz.')
        clean = nh3.clean(comment, tags={'b', 'em', 'p', 'br'},
                          attributes={}, strip_comments=True)
        plain = comment  # Jinja escapes this HTML text context.
    return render_template_string(
        RESULT, mode=app.config['LAB_MODE'], name=name, age=age,
        plain=plain, rich=Markup(clean), clean_source=clean)
```

Nazorat. Oddiy matn plain o‘zgaruvchisida saqlanadi va Jinja orqali kodlanadi. Rich qismiga faqat nh3 tozalagan fragment beriladi. weak rejimida bu ajratish ataylab buzilgan.

1-ilova davomi. C qism — xatolar va ishga tushirish

```
@app.get('/lab-error')
def lab_error():
    raise RuntimeError('A2 controlled diagnostic error')

@app.errorhandler(HTTPException)
def http_error(error):
    return fail('Sorov formati, hajmi, manzili yoki metodi notogri.',
                error.code or 500)

@app.errorhandler(Exception)
def internal_error(error):
    reference = uuid.uuid4().hex[:12]
    app.logger.exception('A2 internal error id=%s', reference)
    return fail('Ichki xato. Qayd raqami: ' + reference, 500)

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--mode', choices=['weak', 'safe'], default='safe')
    args = parser.parse_args()
    app.config['LAB_MODE'] = args.mode
    app.run(host='127.0.0.1', port=8002, debug=False, use_reloader=False)
```

Foydalanilgan manbalar

Mavzu foydalanuvchi taqdim etgan “Veb dasturlar xavfsizligi fani mavzulari.docx” faylidagi A2ga moslandi. Tuzilish A1 amaliy ishi va “Kompyuter_tarmoqlari_A6_1_variant_batafsil.docx” namunasi asosida saqlandi. Rasmiy texnik manbalarga murojaat: 07.09.2026.

[1] OWASP. Input Validation Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[2] MDN. Client-side form validation. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

[3] Python. unicodedata. https://docs.python.org/3/library/unicodedata.html

[4] Flask. Templates. https://flask.palletsprojects.com/en/stable/templating/

[5] nh3. HTML sanitization API. https://nh3.readthedocs.io/en/stable/

[6] OWASP. Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[7] Flask. Handling Application Errors. https://flask.palletsprojects.com/en/stable/errorhandling/